Rating Prediction & Logistic Classification Models for *RentTheRunway* Review Data

Alex Hayslip
Halıcıoğlu Data Science Institute and Dept of Mathematics
University of California, San Diego
La Jolla, California, USA
alhayslip@ucsd.edu

1 INTRODUCTION

Imagine this scenario: you find out that your best friend is getting married, and you receive an invitation to their wedding. Being excited for your best friend and their significant other is not hard. However, many ponder a common question: "What should I wear?" *RentTheRunway* allows users to rent or purchase outfits used for special occasions, weddings, and everyday activities, among other functions. With the help of *RentTheRunway*, we can remove the stress associated with choosing our outfit for this special occasion and focus our energy on the most critical activity: being there for those who matter the most to us.

To build the recommender systems referenced in this report, I used a dataset of user reviews submitted by individuals who purchased an outfit from *RentTheRunway* [1]. This data set has a list data type and returns 192,544 records. This dataset consists of unique features for each user, such as their weight, body type, product category, height, size, age, and bust size. Additionally, columns such as user_id and item_id are created to ensure that each user has a unique ID and that each item has an item unique to that product. The category column connects each product with the product type to which it belongs. Lastly, the column rented for indicates the kind of occasion that influenced the user to purchase an item from *RentTheRunway*.

I approached this scenario by implementing a binary class logistic classification model with the x variable consisting of a feature vector, the user's weight, height, size, age, and bust size. These x variables will predict the y value of whether the user's outfit fits appropriately. These models will be judged by their accuracy score. The accuracy score will be computed by using the balanced_accuracy_score model from the sklearn.metrics library to compute the accuracy from the ytest and prediction values.

For regression tasks, I will start by implementing a baseline model of predicting the rating of the user by using the length of the text in the review process as the x variable. This will indicate the y variable of the rating provided by the user. In successive implementations, I will adjust the parameters of the x variable by building a bag of word models that remove needless parameters like capitalization, stop words, and punctuation from the review text column. I also attempt to predict the ratings by generating the TF-IDF scores from the bag of words model.

1.1 Data Cleaning Procedures

Upon reviewing the *RentTheRunway* review data, I noticed that the only column captured as an integer data type is the size column.

```
]: data[0]

]: {'fit': 'fit',
    'user_id': '420272',
    'bust size': '34d',
    'item_id': '2260466',
    'weight': '137lbs',
    'rating': '10',
    'rented for': 'vacation',
    'review_text': "An adorable romper! Belt and zipper were a little
   had pockets, but other than that-- absolutely perfect! I got a mil
    'body type': 'hourglass',
    'review_summary': 'So many compliments!',
    'category': 'romper',
    'height': '5\' 8"',
    'size': 14,
```

To start this process, I converted the data from a list to a pandas data frame by using pd.DataFrame.

```
[1312]: import pandas as pd

[1313]: data2 = pd.DataFrame(data)

[1314]: data2
```

My first actionable item in cleaning the data was removing any records with null or duplicate values. The total number of records decreased from 146,381 to 146,237.

```
[12]: data2 = data2.dropna()

[13]: len(data2)

[13]: 146381

[14]: #check for duplicate values, there are 144 duplicate records

[15]: s = data2.duplicated()

[16]: s.value_counts()

[16]: False    146237
      True        144
      Name: count, dtype: int64

[17]: data2 = data2.drop_duplicates()

[18]: len(data2)

[18]: 146237
```

I focused my data cleaning efforts on converting the columns of bust size, weight, rating, age, and height into integer values that can be used in the classification model.

To convert the age and rating columns from object values to integer values, I mapped the columns of age and rating to astype(int) to convert these columns into the correct data types.

```
[1324]: #convert age and rating columns to integer values

[1325]: data2['age'] = data2['age'].astype(int)

[1326]: data2['rating'] = data2['rating'].astype(int)
```

To convert the columns bust size and weight from object data types to integer data types, I had to remove the unnecessary string values from both columns. I used list

comprehension to remove the unnecessary string values. Then, I converted the data type for both columns into integer data types.

```
[1328]: data2["weight"] = data2["weight"].str[:-3]
[1329]: data2['weight'] = data2['weight'].astype(int)
[1330]: #convert bust size into an integer, remove string value, then convert into integer
[1331]: data2["bust size"] = data2["bust size"].str[:2]
[1332]: data2['bust size'] = data2['bust size'].astype(int)
```

I decided to convert the data type of the height column into an integer that could be used for the classification model. The height data for each user is captured as an aggregation of their height in feet and inches. I split the height column into two separate columns, feet, and inches, and removed the subsequent string values for both columns. I converted both columns into integer types as well. Additionally, I created a new column called total inches, which multiplied the value in the feet column by twelve and added the value returned in the inches column. As a result of this process, I created a new column called "tot inches," which is viable for a classification model.

```
[1333]: #split height into feet and inches, then create a column that is "total inches"
[1334]: data2[['Feet', 'Inches']] = data2['height'].str.split(' ', expand=True)
[1335]: data2["Feet"] = data2["Feet"].str[:-1]
[1336]: data2['Feet'] = data2['Feet'].astype(int)
[1337]: data2["Inches"] = data2["Inches"].str[:-1]
[1338]: data2['Inches'] = data2['Inches'].astype(int)
[1339]: data2["tot_inches"] = data2["Feet"]*12 + data2["Inches"]
```

Upon initial data exploration, I noticed a few instances of bad data, which contained 0 for the columns of age and size. I dropped these records to ensure data integrity and quality. After making these changes, the total number of records dropped from 146,237 to 145,779.

```
[1344]: #remove records from the dataset where age and size are 0
[1345]: data2.drop(data2[data2["age"] == 0].index, inplace=True)
[1346]: len(data2)
[1346]: 146216
[1347]: #remove records from dataset where size is 0
[1348]: data2.drop(data2[data2["size"] == 0].index, inplace=True)
[1349]: len(data2)
[1349]: 145779
```

Lastly, I built a label encoder to create a new "encoder fit" column on the existing fit column. The fit column contains three values: fit, small, and large. I encoded the new column encoded fit to return 0 for the record of small, 1 for the fit, and 2 for the large record. I made this change as I was interested in performing a multi-class classification model on the y variable of fit. To run this model, I must transform the fit column to generate more than two possible outcomes.

```
[1341]: #build a specific label encoder for the fit column
[1342]: encoding_map = {"small": 0, "fit": 1, "large": 2}
[1343]: data2['encoded fit'] = data2['fit'].map(encoding_map)
```

1.2 Observations from Initial Data Exploration

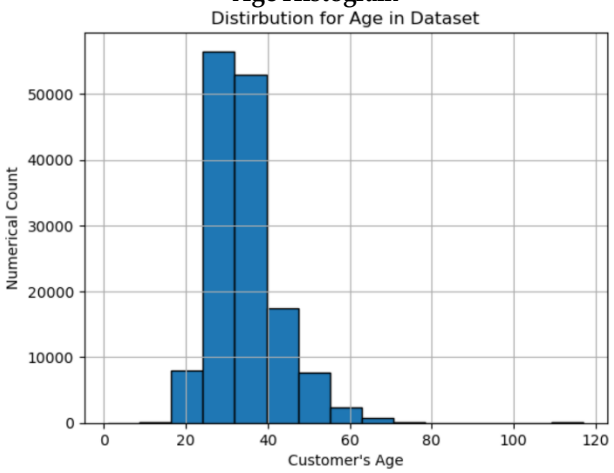
I generated a pivot table with the average age, rating size, and total inches columns grouped by each size (large, fit, and small). I learned that each customer's average age is generally between 33 and 34, regardless of size. On average, the rating for each user is almost one point higher if the user indicates that their outfit fits. I am surprised that users aren't more critical of products classified as small or large.

[1392]:

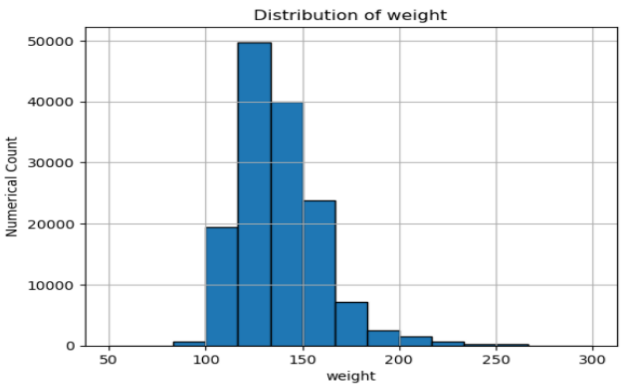
fit	age	rating	size	tot_inches
large	33.8	8.5	10.0	65.2
fit	34.1	9.3	11.3	65.3
small	34.3	8.4	13.7	65.3

I also decided to generate age, size, and weight column histograms with a bin width 15 for each histogram. I was curious to see the distribution composition for these histograms to assess if the data is skewed or follows a normal distribution. I noticed that the age and weight column data distribution is slightly skewed to the right. This information tells me I won't need to perform any log transformation to ensure the data follows a normal distribution.

Age Histogram

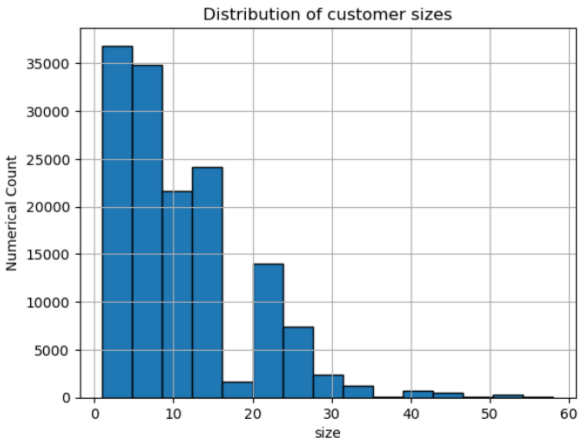


Distribution of Weight



When I generated the size column histogram, I learned that the distribution was skewed to the right. Because of this, I intend to construct a logistic classification model with a transformation of the size column to match an approximately normal distribution.

Distribution of Size



I also used the value counts code on the ratings, rented for, and fit columns. I wanted to see the distribution of records for these columns.

Each review is a unique number for the rating columns within a range of one to ten. I noticed that most of the users provided a positive review. Using subjective reasoning, I defined a positive review as a rating of eight or higher. I learned that 134,480 users out of 145,779 (92.24%) provided a positive review. Additionally, I noticed that 93,869 users out of 145,779 (64.39%) left a rating that contained a perfect score of ten out of ten. Considering that the pivot table showcased a solid average rating per size, I am not surprised that the ratings were overwhelmingly positive. However, I am concerned that the ratings are imbalanced due to a lack of negative feedback.

```
[1369]: data2['rating'].value_counts().head(10)
```

```
[1369]: rating
        10    93867
        8     40613
        6      8322
        4      2196
        2       781
        Name: count, dtype: int64
```

```
[1399]: perfect = (93869)/145779
        print(perfect)
```

```
        0.6439130464607385
```

```
[1400]: positive = (93867+40613)/145779
        print(positive)
```

```
        0.9224922656898455
```

Additionally, I learned that the value counts for the fit column returned 107,239 records out of 145,779, where the user indicated that their outfit fit. This accounts for a percentage of 73.56%. This distribution is concerning as the distribution for these columns is overwhelmingly imbalanced. I intended to use this column as the predictor value for the logistic regression models, so I will need to implement strategies to balance the class weight for the values in this column.

```
[1372]: fit
        fit      107239
        small     19594
        large     18946
        Name: count, dtype: int64
```

```
[1401]: productfit = 107239/145779
        print(productfit)
```

```
        0.7356272165401052
```

Lastly, I wanted to learn more about the different occasions users have rented outfits for. I learned that weddings are the most popular reason users have provided to rent a product. Of 145779 rentals, 42406 are for weddings, which accounts for 29.08%. I do not find this surprising as many users could be interested in renting an outfit temporarily, such as attending a wedding.

```
[1402]: data2['rented for'].value_counts()
```

```
[1402]: rented for
        wedding           42406
        formal affair     30247
        party             27143
        everyday          13248
        work              12113
        other             11474
        date               5922
        vacation           3225
        party: cocktail       1
        Name: count, dtype: int64
```

```
[1403]: wedding = (42406)/145779
        print(wedding)
```

```
        0.2908923781889024
```

2. Identify predictive tasks for analysis

The first predictive task I will perform is a binary classification problem to predict whether the user's outfit fits based on the user's information. The y value in this analysis is the fit column, consisting of binary values (yes or no) regarding whether the users indicated their outfits fit. This baseline model determines how effectively a logistic classification model can predict whether a user suggests their outfit fits. The x values in this analysis are the feature vector and the bust size, weight, age, size, and total inches columns.

2.1 Features for Logistic Classification

The bust size column indicates the consumer's bust size. As described in the previous section, I needed to remove the string value from the bust size column. After this, I used the astype(int) argument to convert the data type for the bust size column from object to integer.

The age column indicates the consumer's age in numerical form. As described in the previous section, I mapped the age column in the dataset to the astype(int) argument to convert the data type from object to integer.

The weight column indicates the customers' weight in pounds. As described in the previous section, I needed to

remove the string value from the weight column. After this, I used the astype(int) argument to convert the data type for the weight column from object to integer.

The total inches column was constructed using the height column as a template. The first step was to split the height column into two separate columns titled feet and inches. After this, I removed the unnecessary string values from the feet and inches columns. Lastly, the total inches column was defined as multiplying the values from the feet column by twelve and adding that number to the data in the inches column.

Lastly, the rating's y value was converted from an object data type to an integer using the astype(int) argument.

As the previous section shows, 107,239 records out of 145,779 indicated their outfit fit. To navigate the issue with the fit column's imbalanced values, the logistic classification model will include the following argument when building the model: class_weight="balanced". Additionally, the balanced_accuracy_score from the sklearn.metrics library will be leveraged to calculate the accuracy score from the ytest and predicted values. The balanced accuracy score generates the accuracy score while accounting for the issue with the imbalanced values in the fit column.

2.2 Baseline Model for Logistic Classification

I built the baseline model by defining the x and y features as I previously stated and composing the train, test, and valid splits as 80%, 10%, and 10%. The baseline model returns an accuracy percentage of 51.95%, slightly better than randomly guessing. However, this baseline model is a weak predictor of whether the user indicated that their outfit fit.

2.3 Features for Logistic Regression

The second predictive task I will analyze is a logistic regression model that uses the features vector and the length of the user reviews to predict the y value of the numeric rating left by the user. The effectiveness of this model will be measured by the MSE value generated from the predicted values and the ytest values.

The review text column is technically an object datatype. However, using the length argument for the review text column, I can generate the numeric value of each review's length in a summation of all characters. The y value of the rating is a numeric value that indicates the user rating that they provided the product on a scale of 0 to 10.

2.4 Baseline Model for Logistic Regression

The following steps were performed to generate a baseline model from the x values of the features vector, the length of the user reviews, and the y value of the rating. I learned that the MSE value for the baseline logistic regression model is 2.847395618969223. This is very high, indicating that the baseline model has a weaker predictive accuracy of the ratings.

3.1 Logistic Classification Model

To improve the model's performance compared to the baseline, I have decided to experiment with different types of feature engineering, such as adjusting the train test split distribution, implementing a standard scaler transformation on the x variables, removing the outliers from the size and age columns, removing the column of age from features represented in the x variable, and performing a logs transformation on the size column.

3.1.1 Logistic Classification Model with adjustment to Train Test Validation split

The baseline logistic classification model consisted of an 80%, 10%, and 10% split of the training, testing, and validation data. Out of 145,779 possible records, 116,623 belonged to the training data, 14,578 were classified as validation data, and 14,578 were labeled as test data. The objective of computing a split on the training, testing, and validation data is to prevent overfitting, where the model is much better on the training set than the testing and validation sets.

I adjusted these parameters by classifying 70% of the data as training data, 15% as validation data, and 15% as test data. I was convinced this adjustment would improve the new model's accuracy score compared to the baseline model. After performing manual computations, I learned that 70% of the training data accounts for 102,045 records, 15% of the validation data returns 21,867 records and 15% of the test data returns 21,867 records. I defined these as the new variables Ntrain, Nvalid and Ntest.

After performing the logistic classification model with the new Ntrain, Nvalid and Ntest variables, I learned that the accuracy score increased to 52.15%.

Adjusting the training, test, and validation data distribution improved the accuracy score compared to the baseline logistic regression model. I will use the new train, test, and validation data distribution on the rest of the logistic classification models. However, the accuracy score is still a weak predictor of whether the user's outfit fits based on the x variables.

3.1.2 Logistic Classification Model with Standard Scaling of X features

The next feature engineering technique I tried was implementing a logistic classification model in which a standard scaler transformed the x features. By standardizing the x features, the data now follows normal distribution parameters where the mean is 0 and standard deviations are 1. This adjustment ensures that no feature would disproportionally influence the algorithm's performance due to scalability issues. I wanted to leverage the standard scaler library on the x features to see if the model would perform better.

```
[92]: Ntrain = 102045
      NValid = 21867
      Ntest = 21867

[93]: Xtrain = X[:Ntrain]
      ytrain = y[:Ntrain]
      Xtest = X[:Ntest]
      ytest = y[:Ntest]
      Xvalid = X[:NValid]
      yvalid = y[:NValid]

[94]: def feature(datum):
          a = datum["bust size"]
          b = datum["weight"]
          c = datum["age"]
          d = datum["size"]
          e = datum["tot_inches"]
          return [1, a, b, c, d, e]

[95]: X = [feature(d) for d in dict_list]
      y = [d["fit"] == "fit" for d in dict_list]

[96]: from sklearn.preprocessing import StandardScaler

[97]: sc = StandardScaler()
      Xtrain = sc.fit_transform(Xtrain)
      Xtest = sc.transform(Xtest)

[98]: from sklearn.linear_model import LogisticRegression
      mod = linear_model.LogisticRegression(class_weight="balanced")
      mod.fit(Xtrain, ytrain)

[98]: LogisticRegression
      LogisticRegression(class_weight='balanced')

[99]: predictions = mod.predict(Xtest)
```

After performing these adjustments. I learned that the model returns an accuracy score of 52.49%. The standard scaler adjustments increased the accuracy percentage compared to the baseline model. Additionally, this model performs better than the model that only contained the adjustments to the train test split. I will incorporate the standard scaler into the rest of the logistic classification models.

3.1.3 Logistic Classification Model without outliers

Additionally, I attempted to perform a logistic regression model without the outlier data present in the age and size columns. I intended to see if removing the outliers improved the performance of the logistic classification model.

I created a new data frame, defined the first and third quartiles for the age and size columns, and used the drop feature from the pandas data frame to remove the records classified as outliers. The total number of records was reduced from 145,779 to 135,663.

```
[106]: import numpy as np

[107]: data3 = data2

[108]: Q1 = data3['age'].quantile(0.25)
       Q3 = data3['age'].quantile(0.75)

[109]: IQR = Q3 - Q1

[110]: threshold = 1.5

[111]: outliers = data3[(data3['age'] < Q1 - threshold * IQR) | (data3['age'] > Q3 + threshold * IQR)]

[112]: data3 = data3.drop(outliers.index)

[113]: len(data3)

[113]: 137600

[114]: Q1 = data3['size'].quantile(0.25)
       Q3 = data3['size'].quantile(0.75)

[115]: IQR = Q3 - Q1

[116]: threshold = 1.5

[117]: outliers = data3[(data3['size'] < Q1 - threshold * IQR) | (data3['size'] > Q3 + threshold * IQR)]

[118]: data3 = data3.drop(outliers.index)

[119]: len(data3)

[119]: 135663
```

After making these changes, I implemented the logistic classification with a train, test, and validation distribution of 70%, 15%, and 15%. The training set consisted of 102,045 records, the validation set contained 21,867 records, and the test data set contained 21,867 records. Additionally, the standard scaler features were deployed on the x features.

I learned that this model's accuracy score was 51.60%. This model could have been more effective at increasing its accuracy score. It performed worse than the other two models with feature engineering adjustments and the baseline model.

3.1.4 Logistic Classification Model without the Age Column

I removed the age column from the function of x features before implementing my next logistic classification model. Of all the x features I implemented for the logistic classification model, age was the least effective in determining whether a user's outfit fit.

I redefined the feature function that populates the x variable's features to remove the age column. The features for the x variable used in this model are the feature vector, bust size, weight, size, and total inches. The distribution of train, test, and validation data is the same as other models (102,045, 21,867, 21,867)

```
[136]: Ntrain = 102045
       NValid = 21867
       Ntest = 21867
```

```
[137]: def feature(datum):
           a = datum["bust size"]
           b = datum["weight"]
           c = datum["size"]
           d = datum["tot_inches"]
           return [1, a, b, c, d]
```

```
[138]: X = [feature(d) for d in dict_list]
       y = [d["fit"] == "fit" for d in dict_list]
```

```
[139]: Xtrain = X[:Ntrain]
       ytrain = y[:Ntrain]
       Xtest = X[:Ntest]
       ytest = y[:Ntest]
       Xvalid = X[:NValid]
       yvalid = y[:NValid]
```

```
[140]: sc = StandardScaler()
       Xtrain = sc.fit_transform(Xtrain)
       Xtest = sc.transform(Xtest)
```

I learned that this model's accuracy score is 52.56%. This model returns the highest accuracy score out of all the models deployed. This provides enough evidence to remove age as a column in the x features for the rest of the logistic regression models.

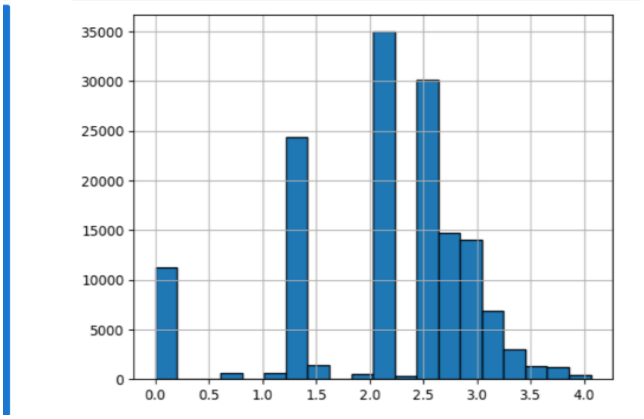
3.1.5 Logistic Classification Model with a natural log transformation

I wanted to transform the size column because the data distribution is skewed to the right. I wanted to see if the accuracy score improved due to using a transformed version of the size column instead of the size column.

I created a new column titled "size_natural_log" by using the np log library on the size column.

```
[1099]: data2["size_natural_log"] = np.log(data2["size"])
```

```
[1100]: plt.hist(data2["size_natural_log"], bins=20, edgecolor="black")
        plt.grid()
        plt.show()
```



After implementing this model with the size natural log as one of the x variables, I noticed that the accuracy score was 51.11%. This model's accuracy score is the worst of all the models that I ran. This proves that transforming the size column does not increase the accuracy. Instead, the accuracy score worsens where the baseline accuracy model is more effective in predicting whether the user's outfit fits.

3.16 Multi-Class Classification

So far, I have constructed binary classification models for logistic classification to predict whether the specified x variables are effective in predicting whether the user's outfit fits. I was interested in determining if the accuracy score for a baseline multi-class classification was more effective than the baseline binary classification model.

The x features of bust size, weight, age, size, and total inches are the same as in the binary classification model. For the y variable, I will use the encoded fit column I created while using the label encoder. The value for fit is encoded as 1, the value for small is encoded as 0, and the value for large is encoded as 2.

After running this model, I learned that the accuracy percentage is 46.64%. The performance of the Multi-Class Classification model is much worse than that of the binary logistic classification model. I was surprised to see the multi-class classification model perform as poorly as it did, as I thought that the model would be more effective in classifying the type of fit value that the customer provided.

```
[155]: Ntrain = 116623
       NValid = 14578
       Ntest = 14578
```

```
[156]: def feature(datum):
           a = datum["bust size"]
           b = datum["weight"]
           c = datum["age"]
           d = datum["size"]
           e = datum["tot_inches"]
           return [1, a, b, c, d, e]
```

```
[157]: X = [feature(d) for d in dict_list]
       y = [d["encoded fit"] for d in dict_list]
```

```
[158]: Xtrain = X[:Ntrain]
       ytrain = y[:Ntrain]
       Xtest = X[:Ntest]
       ytest = y[:Ntest]
       Xvalid = X[:NValid]
       yvalid = y[:NValid]
```

```
[159]: sc = StandardScaler()
       Xtrain = sc.fit_transform(Xtrain)
       Xtest = sc.transform(Xtest)
```

```
[160]: mod = linear_model.LogisticRegression(class_weight="balanced", multi_class="ovr", solver="lbfgs")
       mod.fit(Xtrain, ytrain)
       predictions = mod.predict(Xtest)
```

```
[161]: classaccuracy5= balanced_accuracy_score(ytest, predictions)
```

```
[162]: classaccuracy5
```

3.17 Fine Tuning

Upon further review, the logistic classification model that performed the best was described in subsection 3.1.4. I built a regularization pipeline to optimize the model's performance by iterating for multiple c values and returning the best model's c value, balanced error rate, and accuracy score. The best model returns the following parameters:

C = 0.01; Testing BER = 0.473725560201701, Testing Accuracy Score = 0.526274439798299.

3.18 Model Strengths and Weaknesses

The strength of the logistic regression model is that the composition of x features followed a normal distribution and did not need to be adjusted to optimize the model's performance.

The biggest weakness of this model was that the y value of fit was imbalanced. As previously indicated, 107,239 users out of 145,779 noted that their outfits fit, which accounts

for 73.56% of the data. I attempted to counteract this deficiency by using the class_weight="balanced" argument for each logistic regression model. Additionally, I used the balanced_accuracy_score from the sklearn.metrics library to effectively generate the accuracy score from imbalanced data.

3.2 Logistic Regression Model

To improve the model's performance compared to the baseline, I have decided to experiment with different types of models, such as a bag of words model without capitalization and punctuation, a bag of words model without capitalization, punctuation, and stemming, a model based on the tdidf scores from a bag of words model, and a bag of words model with capitalization, punctuation, and stop words.

3.21 Bag of Words Model for 2000 Most Popular Words (no capitalization or punctuation)

I wanted to optimize the review text column to improve the baseline model's performance. Instead of using the review length as the x variable, I tried to build a bag of words model that returned the sum of the 2000 most popular words in the review.

I built a bag-of-words model that returned the count of each word without punctuation or capitalization.

```
[1174]: from collections import defaultdict
        import string
        from nltk.stem.porter import *

[1175]: wordCount = defaultdict(int)
        totalWords = 0

[1176]: sp = set(string.punctuation)
        stemmer = PorterStemmer()

[1177]: wordCount = defaultdict(int)
        sp = set(string.punctuation)
        for d in dict_list:
          r = ''.join([c for c in d['review_text'].lower() if not c in sp])
          for w in r.split():
            wordCount[w] += 1

        counts = [(wordCount[w], w) for w in wordCount]
        counts.sort()
        counts.reverse()

[1178]: NW =  2000

[1179]: words = [c[1] for c in counts[:NW]]

[1180]: wordId = dict(zip(words, range(NW)))
        wordSet = set(words)

[1181]: def feature(datum):
            bow = [0]*NW
            r = datum['review_text'].lower()
            r = "".join([c for c in r if c not in sp])
            ws = r.split()
            for w in ws:
                if w in wordSet:
                    bow[wordId[w]] += 1
            return bow + [1]
```

When I performed the accuracy, I noticed that the MSE returned a value of 1.7525952348287375, significantly lower than the MSE returned in the logistic regression model. This model proves that the model's accuracy can be improved by transforming the words in the review text column.

3.22 Bag of Words Model for 2000 Most Popular words (no stemming)

I attempted to build on the success of the previous model by creating a bag of words model for the 2000 most popular words. In addition to the last model's adjustments, I attempted to implement stemming in the bag of words model. The goal of stemming is to eliminate prefixes and suffixes for each word to return each word to its original form.

```
[1185]: wordCount = defaultdict(int)
        sp = set(string.punctuation)
        stemmer = PorterStemmer()
        for d in dict_list:
          r = ''.join([c for c in d['review_text'].lower() if not c in sp])
          for w in r.split():
              w = stemmer.stem(w)
              wordCount[w] += 1

        counts = [(wordCount[w], w) for w in wordCount]
        counts.sort()
        counts.reverse()

[1186]: NW =  2000

[1187]: words2 = [c[1] for c in counts[:NW]]

[1188]: wordId2 = dict(zip(words2, range(NW)))
        wordSet2 = set(words2)

[1189]: def feature2(datum):
            bow = [0]*NW
            r = datum['review_text'].lower()
            r = "".join([c for c in r if c not in sp])
            ws = r.split()
            for w in ws:
                if w in wordSet2:
                    bow[wordId2[w]] += 1
            return bow + [1]
```

After implementing the logistic regression model with this bag of words model, I learned that the MSE is 2.107102025883752.

This model performs better than the baseline logistic regression model. However, the MSE is higher than the bag of words model, which only removed capitalization and punctuation. I am surprised that the MSE for this model is higher than that of the previous bag of words model. I thought that the steaming process would result in a lower MSE because words that were deemed to be unnecessary were removed.

3.23 Bag of Words Model for 2000 Most Popular Words with TF-IDF Scores

I wanted to see how effectively a bag of words model with TF-IDF scores would predict the user's rating. I started this scenario by constructing the same bag of words model as seen in 3.2.1.

```
28]: import math

29]: wordCount = defaultdict(int)
     sp = set(string.punctuation)
     for d in dict_list:
       r = ''.join([c for c in d['review_text'].lower() if not c in sp])
       for w in r.split():
         wordCount[w] += 1

30]: counts = [(wordCount[w], w) for w in wordCount]
     counts.sort()
     counts.reverse()

31]: words = [x[1] for x in counts[:2000]]

32]: wordId = dict(zip(words, range(NW)))
     wordSet = set(words)
```

Then, I defined the variables of TF and DF for term frequency and document frequency. The objective of term frequency is to isolate how often each term appears in the document.

```
[233]: df = defaultdict(int)
       for d in dict_list:
           r = ''.join([c for c in d["review_text"].lower() if not c in sp])
           for w in set(r.split()):
               df[w] += 1

[234]: review = dict_list[0]

[235]: tf = defaultdict(int)
       r = ''.join([c for c in review["review_text"].lower() if not c in sp])
       for w in r.split():
           tf[w] = 1

       tfidf = dict(zip(words,[tf[w] * math.log10(len(dict_list) / df[w]) for w in words]))
       tfidfquery = [tf[w] * math.log10(len(dict_list) / df[w]) for w in words]

[236]: maxTf = [(tf[w],w) for w in words]
       maxTf.sort(reverse=True)
       maxTfIdf = [(tfidf[w],w) for w in words]
       maxTfIdf.sort(reverse=True)

[237]: l = []
       for w in words:
           idf = math.log10(len(dataTrain) / df[w])
           t = tf[w]
           l.append((idf, t*idf))

[238]: #generate Logistic Regression Model of tdf scores

[239]: from scipy.sparse import lil_matrix

[240]: def feature(datum):
           feat = [0]*len(words)
           r = ''.join([c for c in datum['review_text'].lower() if not c in sp])
           tokens = r.split()
           for w in tokens:
               if w in wordSet:
                   feat[wordId[w]] += 1 * math.log10(len(dataTrain) / df[w])
           feat.append(1) #offset
           return feat

[241]: nf = len(feature(dict_list[0]))

[242]: X = lil_matrix((len(dict_list), nf))

[243]: for i in range(len(dict_list)):
           if not (i % 2000):
               print(i)
           x = feature(dict_list[i])
           for j in range(nf):
               if x[j]:
                   X[i,j] = x[j]
```

I learned that the MSE value from the TF-IDF scores model returns as 1.6150363561531074. This model outperforms the baseline model and performs better than the other logistic regression models I have tried.

3.24 Bag of Words Model for 2000 Most Popular Words (no stop words)

I wanted to expand upon the previously built model in 3.2.1 by removing the stop words, capitalization, and stop words. Stop words like are or too do not provide meaningful information. I wanted to see if I could improve the MSE in the bag of words model by removing the stop words

```
[1218]: wordCount = defaultdict(int)
        sp = set(string.punctuation)
        for d in dict_list:
          r = ''.join([c for c in d['review_text'].lower() if not c in sp])
          for w in r.split():
              if w not in stop_words:
                  wordCount[w] += 1

        counts = [(wordCount[w], w) for w in wordCount]
        counts.sort()
        counts.reverse()

[1219]: words4 = [x[1] for x in counts[:2000]]

[1220]: wordId4 = dict(zip(words4, range(NW)))
        wordSet4 = set(words4)

[1221]: def feature4(datum):
            bow = [0]*NW
            r = datum['review_text'].lower()
            r = "".join([c for c in r if c not in sp])
            ws = r.split()
            for w in ws:
                if w in wordSet4:
                    bow[wordId4[w]] += 1
            return bow + [1]
```

I learned that the MSE value returns a value of 1.7372296154022042 when the stop words are removed. Compared to the other models, it outperforms the baseline and bag of words models listed in 3.2.1 and 3.2.2. However, this model is less effective than the TD-IDF model in improving the MSE score.

3.25 Fine Tuning

Upon further review, the logistic classification model that performed the best was described in subsection 3.23. I built a regularization pipeline using the same model for TF-IDF scores by iterating for multiple c values and returning the c value and MSE score of the best model. The best model returns the following parameters: C=100; Testing MSE= 1.6130241917043948.

3.26 Strengths and weaknesses of the model

The biggest strength of the logistic regression model is that it was easy to implement and subsequently adjust with different models. Each adjustment to the logistic regression was an improvement as compared to the baseline model. The computational time for each model was efficient even with the complexity of bag of words models that were introduced.

However, the logistic regression model's biggest weakness is its assumption that the independent x and y variables have a linear relationship. Using logistic regression, analyzing more complex relationships between the x and y variables is impossible.

4. Review of Relevant Academic Literature

I used the *RentTheRunway* dataset to measure clothing fit for users who provided a review after purchasing a product from *RentTheRunway*. Professor McAuley's research team implemented this dataset and the ModCloth dataset to analyze the relationship between the customer's measurements and whether the product fits the user. This analysis also includes other variables that impact whether the customer indicates that their outfit fits, such as the consumer's style or preferences. Professor McAuley's team treated this conundrum as a regression problem by deploying an ordinal regression model. This model was initially built by generating a latent factor model to understand the fit semantics and produce a score that assesses the fit of a particular product a customer purchased. This model was fine-tuned by implementing a Hinge loss

function to follow an ordinal regression problem. The conclusions generated from this paper were that a solution consisting of a latent factor module to distinguish the fit semantics and a metric learning approach to counteracting the label imbalance issues successfully navigated the problems with the imbalance in the fit column. [1]

A research team from the Computer Vision Foundation attempted to create a unique solution to analyze the problem of making a recommender system that accounts for the customer's body type. The researchers developed an encoding algorithm called ViBE that captures the similarity between the user's body type and the recommended clothing items. This strategy was constructed to eliminate the one-size-fits-all approach to fashion. The dataset was curated using images of people with diverse body shapes, full body photos used to estimate the body shapes, and a rating that generates whether the garment is compatible with the person's body type. The dataset is trained using K-Means clustering and nearest neighbors to develop five unique body shapes. Two recommender systems were created from this data: a body-agnostic recommender system that does not utilize the user's body shape and item parameters and a body-aware recommender system that uses the customer's body features and clothing visual features. As the result of testing the performance of both recommender systems on quantitative and human volunteers, the research team concluded that the body-aware recommender system could generate more effective recommendations than the body-agnostic recommender systems [2].

A research team assembled by eBay attempted to analyze the problem of predicting item size using a deep learning framework and transformers. This strategy accounts for size ambiguity, such as different sizing schemes in respective countries and different scales for certain products—the model PreSize is built to predict size as part of a multi-classification problem.

The framework of this model initially starts with item embedding, which generates a feature representation for each item in the buyer's purchase history. The second step was to obtain a representation of the buyers' purchase history. This step introduces temporal embeddings to ensure the model knows the ordering of past purchases and the time-based relevancy. The next step is to introduce context features representation for the target item by adjusting the size preferences of a given buyer that fits a particular target item. Lastly, the classifier obtains the size predictions by concatenating the buyer's purchase history and context embeddings. The preSize algorithm generates a micro precision, macro precision, macro rell, and macro F1 score that significantly outperforms all baseline models. [3].

Like the approach in the first paper, I also recognized the issue of determining whether the product fits based on the customer's measurements. I also tried to account for the imbalanced values in the fit column. However, I did not think of implementing a regression model to assess the fit of an item purchased by a customer based on their measurements. When building each logistic classification model, I treated this scenario as a logistic classification problem and attempted to balance the class weights using the class_weights="balanced" argument. Additionally, I used the balanced_accuracy_score module from the sklearn.metrics library. My solution failed as I could not generate a viable logistic classification even with the adjustments to account for the imbalanced data in the fit column.

The second paper attempted to use a recommender system to provide an accurate clothing recommendation for the user based on their body type and the clothing's visual features. I did not consider using a recommender system to analyze the fit type values in the *RentTheRunway* dataset. However, I considered performing a logistic classification model with the customer's measurements as the x features and the y value of body type. Specifically, I considered using the label encoder to generate a new column to map each body type to a unique number. I wanted to see if the logistic classification model would be more effective in predicting the user's body type than in predicting whether the customer's outfit fit. I decided to forgo this idea due to the volume of logistic classification models already committed in the earlier sections. However, I would like to perform this analysis separately to see if the logistic classification model effectively predicts the encoded column for the customer's body type from the customer's measurements.

The third paper reduced the ambiguity of the size column presented in the eBay fashion purchases dataset from June 2019 to June 2020 by implementing transformers on the size column. I built a logistic classification model on the *RentTheRunWay* data by taking the customer's features as x features to predict the y value of whether the customer's outfit fits. I performed a natural log adjustment on the size column as I determined the size column did not follow a normal distribution. I implemented the logistic classification model with the x features, including the natural log of size as a part of the x features. I thought that the performance of the logistic classification model would improve as the natural log of the size column followed an approximately normal distribution. However, I learned that the accuracy score for the logistic classification model with the natural log adjustment for the size column returned the worst accuracy score out of all the models I tested. I did not consider the possibility of implementing a logistic classification model to predict the customer's size from their measurements. I am interested in implementing a multi-class classification model to see how effectively I predict the customer's size from their measurements.

5. Results

The logistic classification model proved ineffective in predicting whether the user's outfit fit them when using the x variables provided. The y value, represented by the fit column, was plagued by an imbalanced data composition. In every logistic classification model, I used the class_weight="balanced" argument in addition to the balanced_acuracy_score that is imported from the sklearn.metrics library. However, these adjustments were ineffective in combating the imbalanced data for the fit column, as displayed by the accuracy score of each model.

I attempted to make iterative adjustments to the baseline model, such as adjusting the train test split, implementing a standard scaler, removing the age column, and removing the outliers for the size and age columns. Adjustments such as changing the distribution of the train, test, and validation data, removing the age column, and implementing a standard scaler provided incremental improvements in the accuracy score compared to the baseline model. However, the logistic classification could be more

effective, as evidenced by the accuracy score for each model.

Conversely, the logistic regression model proved to be an effective method of predicting the user's rating from the x variable of the review text column. Each successive bag of words model generated an MSE that outperformed the baseline logistic regression model by a substantial margin. The most successful logistic regression model was defined by compiling a bag of words model with the TF-IDF scores for the 2000 most popular words, creating a sparse matrix of the x variables, and computing the MSE.

5.1 Logistic Classification Models

Please refer to the following table to summarize the logistic classification models used and their respective scores.

Table 1 Accuracy Scores by Logistic Classification Model

Model Type	Test Accuracy	Split
Baseline	0.5195	80/10/10
Train/Test/Split	0.5251	70/15/15
Standard Scaler	0.5249	70/15/15
Remove Outliers	0.5160	70/15/15
Remove Age	0.5256	70/15/15
Natural Log	0.5151	70/15/15
Multi-Class Classification	0.4464	80/10/10

The best model is the logistic classification model, where the age column is removed from the x features. I had a hunch that the age column was not nearly as predictive of whether the user's outfit fit compared to the other features implemented as x variables, which influenced me to make that change.

I was encouraged to see that the model outperformed the baseline model, which included standard scaling on the x features. I prefer implementing a standard scaler on classification models to ensure that the x features follow a normal distribution as closely as possible. The standard scaler is especially helpful when the x variables have their unique context. For instance, the age and size columns are both numerical values, but they have differing values for their mean, median, first quartile, and third quartile.

I was surprised that the model performed worse than the baseline after removing the outliers from the age and size columns. I hypothesized that eliminating irrelevant data from the age and size columns would increase the accuracy score by removing insufficient data from the analysis. However, it had the opposite effect: it performed worse than I expected and worse than the baseline logistic classification model.

5.2 Hyperparameter Tuning of Logistic Classification Models

I performed hyperparameter tuning on the logistic classification model where the age column was removed from the x features to return the best c value, balanced error rate, and test accuracy score. I built a pipeline function to iterate over the c values of 0.0001, 0.001, 0.01, 0.1, 1, 10, 100, 1000, and 10000, as well as the training and test data set.

Please refer to the table below to see the performance of the best logistic classification model when looped with all the c values that were previously referenced in the regularization pipeline.

Table 2 Results of Regularization Pipeline for Best Logistic Classification Model

C Value	Test Balanced Error Rate	Test Accuracy Score
0.0001	0.4813985880674807	0.5186014119325193
0.001	0.4749831400172263	0.5250168599827737
0.01	0.473725560201701	0.526274439798299
0.1	0.47440494068996697	0.525595059310033
1	0.47443622588581225	0.5255637741141878
10	0.47443622588581225	0.5255637741141878
100	0.47443622588581225	0.5255637741141878
1000	0.47443622588581225	0.5255637741141878
10000	0.47443622588581225	0.5255637741141878

The testing accuracy score increases each iteration for the C value of 0.0001 to 0.01. The best model returns a C value of 0.01, a balanced error rate of 0.473725560201701, and an accuracy score of 0.526274439798299. From the C value of 0.1 to the C value of 10000, the accuracy score flatlines at roughly 0.5255.

5.3 Performance of the Logistic Regression Models

Please refer to the following table, which showcases the performance of each logistic regression model.

Table 3 MSE Values by Logistic Regression Model

Model Type	Test MSE Value	Split
Logistic Regression Baseline	2.8473	70/15/15
Bag of Words	1.7525	70/15/15
Bag of Words without stemming	2.1071	70/15/15
Bag of Words with TF-IDF scores	1.6150	70/15/15
Bag of words no stop words	1.7372	70/15/15

The best model among the logistic regression models is the model that appended the TF-IDF scores to the initial bag of words model for the 2000 words that occurred most frequently. This should not be a surprise, as the term frequency (TF) model highlights the importance of certain words within a document by illustrating how often they appear. By computing the TD-IDF scores, I can compile a sparse matrix to see how important the collection of words is to how the user rates the product.

I was surprised to learn that the logistic regression model, which contained the stemming, performed better than the initial bag of words model and the bag of words model without the stop words. The bag of words model without stemming performed significantly better than the MSE generated by the baseline logistic regression model. However, the MSE would be comparable to the other bag of word models because it removes unnecessary words related to a common word stem.

Any of these iterations of the logistic regression model would be acceptable as they generate a significantly lower MSE than the baseline model. This proves that there were plenty of options to showcase how the review_text column can predict the user's rating.

5.4 Hyperparameter Tuning of Logistic Regression Models

I performed hyperparameter tuning on the logistic

regression model, where the TF-IDF scores were constructed from the bag of words model. I built a pipeline function to iterate over the c values of 0.0001, 0.001, 0.01, 0.1, 1, 10, 100, 1000, and 10000 and the training and test data set to return the MSE value for each iteration.

Table 4 Results of Regularization Pipeline for Best Logistic Regression Model

C Value	Test MSE Value
0.0001	2.7116659807015138
0.001	2.186308135546714
0.01	1.8367402935930854
0.1	1.6657063154525082
1	1.6150363561531074
10	1.6132071157451868
100	1.6130241917043948
1000	1.622902089907166
10000	1.6313165957836009

The testing MSE score increases each iteration for the C value of 0.0001 to 100. The best model returns a C value of 100 and a test MSE value of 1.6130241917043948. From the C value of 1000 to the C value of 10000, the MSE score slightly increases.

6 ACKNOWLEDGMENTS

Thank you to Professor McAuley and Shashi Dhanasekar for their efforts in teaching this course. Additionally, thank you to everyone who reviewed this report and provided feedback.

REFERENCES

[1] Rishabh Misra, Mengting Wan, and Julian McAuley. 2018. Decomposing fit semantics for product size recommendation in metric spaces. *Proceedings of the 12th ACM Conference on Recommender Systems* (September 2018). DOI:http://dx.doi.org/10.1145/3240323.3240398

[2] Wei-Lin Hsiao and Kristen Grauman. 2020. Vibe: Dressing for diverse body shapes. *2020 IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)* (June 2020). DOI:http://dx.doi.org/10.1109/cvpr42600.2020.01107

[3] Yotam Eshel, Or Levi, Haggai Roitman, and Alex Nus. 2021. Presize: Predicting size in e-commerce using transformers. *Proceedings of the 44th International ACM SIGIR Conference on Research and Development in Information Retrieval* (July 2021). DOI:http://dx.doi.org/10.1145/3404835.3462844